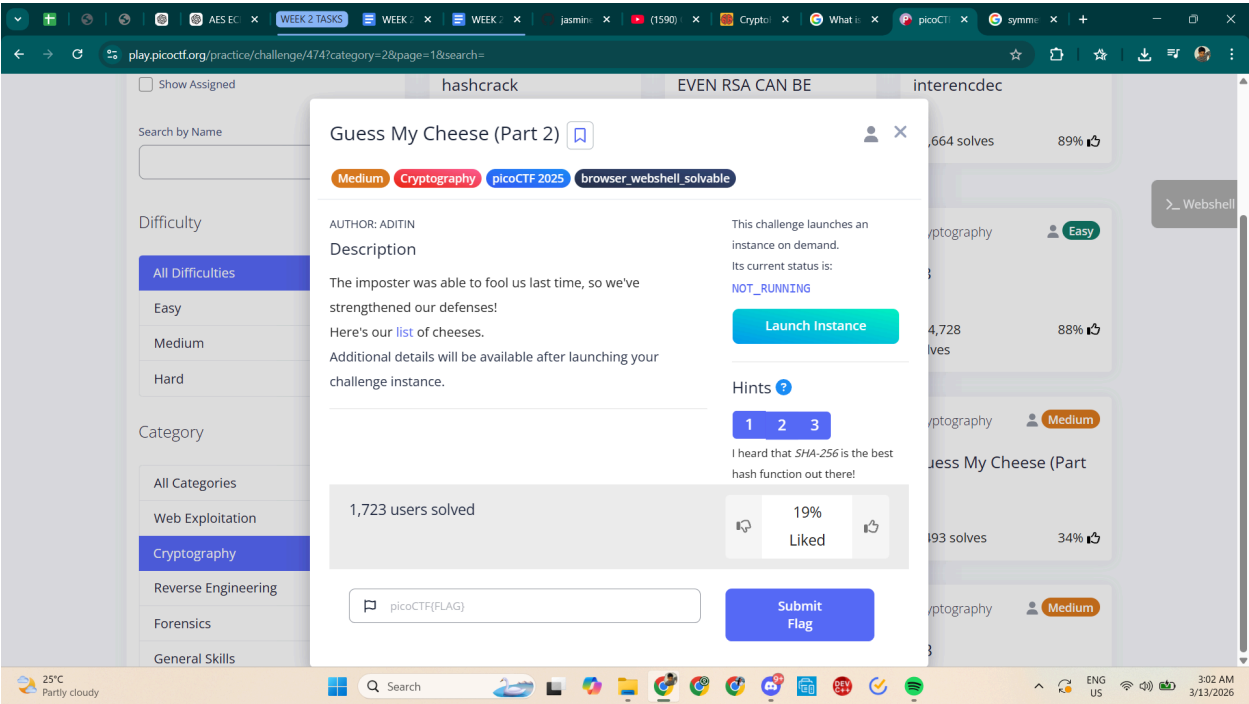


## PRACTICAL ACTIVITY 4: Solving a Symmetric-Cryptography CTF Challenge



CHALLENGE TITLE: **Guess My Cheese Part 2**

(<https://play.picoctf.org/practice/challenge/474?page=1&search=guess%20my%20cheese%20part%202>)

PLATFORM: **picoCTF**

CATEGORY: **Cryptography (Medium)**

### Objective:

To solve a cryptography-based Capture the Flag (CTF) challenge involving the SHA-256 hashing algorithm by applying concepts such as hashing, salting, and rainbow tables to recover the original input used to generate a given hash.

### Challenge Overview:

The Guess My Cheese Part 2 challenge required identifying the correct cheese name and salt value used to generate a SHA-256 hash. The challenge description hinted that the system applied two hexadecimal nibbles of salt to the cheese name before hashing.

A nibble represents half a byte, equivalent to a single hexadecimal character. Therefore, two nibbles correspond to two hexadecimal characters, giving 256 possible salt combinations. The server would display a SHA-256 hash, and the objective was to determine which cheese from a provided list and which salt value produced that hash.

PROCEDURE:

1. Analyzing the Challenge Hints

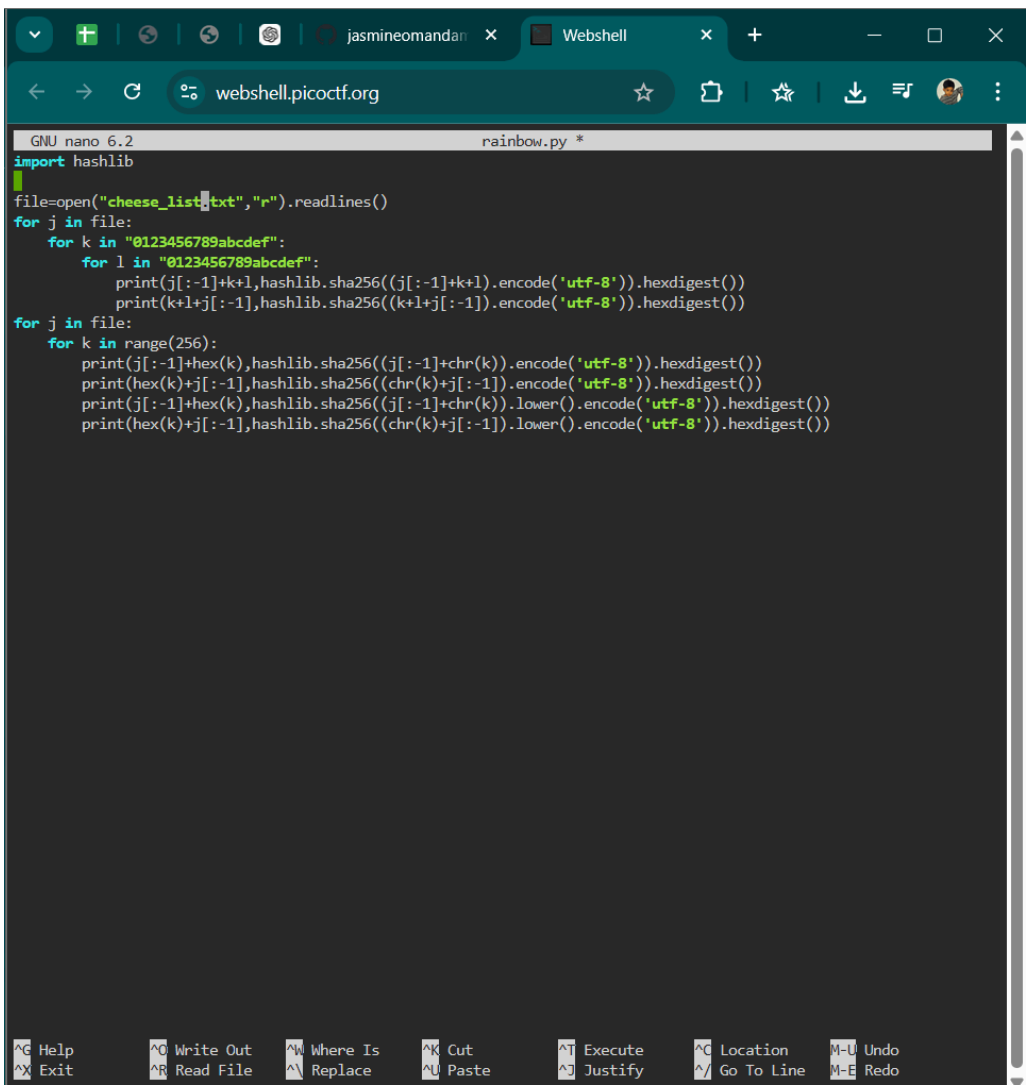
The hints suggested several important details:

- The hashing algorithm used was **SHA-256**.
- The system added **two hexadecimal characters of salt**.
- The cheese names came from a **known list of cheeses**.
- Rainbow tables could potentially be useful for solving the challenge.

From these hints, it became clear that the most effective approach would be to **precompute hashes for all possible combinations of cheeses and salts**, then compare them with the hash provided by the server.

2. Creating a Rainbow Table

To automate this process, I created a Python script named **rainbow.py**. The script used the **hashlib** library to compute SHA-256 hashes.



```
GNU nano 6.2 rainbow.py *
import hashlib

file=open("cheese_list.txt","r").readlines()
for j in file:
    for k in "0123456789abcdef":
        for l in "0123456789abcdef":
            print(j[:-1]+k+l,hashlib.sha256((j[:-1]+k+l).encode('utf-8')).hexdigest())
            print(k+l+j[:-1],hashlib.sha256((k+l+j[:-1]).encode('utf-8')).hexdigest())
for j in file:
    for k in range(256):
        print(j[:-1]+hex(k),hashlib.sha256((j[:-1]+chr(k)).encode('utf-8')).hexdigest())
        print(hex(k)+j[:-1],hashlib.sha256((chr(k)+j[:-1]).encode('utf-8')).hexdigest())
        print(j[:-1]+hex(k),hashlib.sha256((j[:-1]+chr(k)).lower().encode('utf-8')).hexdigest())
        print(hex(k)+j[:-1],hashlib.sha256((chr(k)+j[:-1]).lower().encode('utf-8')).hexdigest())
```

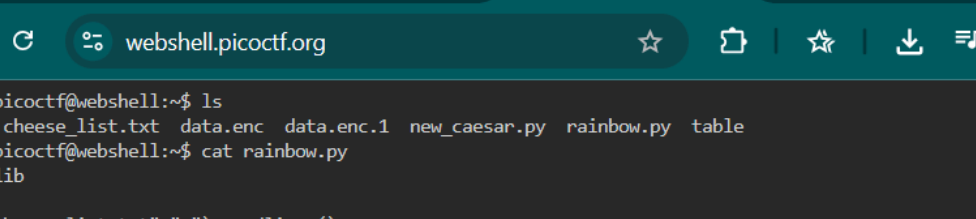
The program performed the following steps:

1. Read a file containing the list of cheeses.
2. Generate all possible two-character hexadecimal salt combinations.
3. Append each salt value to each cheese name.
4. Convert the string to lowercase to match the hashing format used by the server.
5. Compute the SHA-256 hash.
6. Store the hash and corresponding cheese-salt combination.

The script essentially generated a **rainbow table**, which maps hashes back to their original inputs.

### 3. Generating the Rainbow Table

I executed the script with the following command: ***python rainbow.py > table***



The screenshot shows a web browser window with the address bar displaying 'webshell.picotf.org'. The browser's address bar and tabs are visible at the top. The main content area shows a terminal window with the following text:

```
REIAHNEVIL-picotf@webshell:~$ ls
README.txt cheese_list.txt data.enc data.enc.1 new_caesar.py rainbow.py table
REIAHNEVIL-picotf@webshell:~$ cat rainbow.py
import hashlib

file=open("cheese_list.txt","r").readlines()
for j in file:
    for k in "0123456789abcdef":
        for l in "0123456789abcdef":
            print(j[:-1]+k+l,hashlib.sha256((j[:-1]+k+l).encode('utf-8')).hexdigest())
            print(k+l+j[:-1],hashlib.sha256((k+l+j[:-1]).encode('utf-8')).hexdigest())
for j in file:
    for k in range(256):
        print(j[:-1]+hex(k),hashlib.sha256((j[:-1]+chr(k)).encode('utf-8')).hexdigest())
        print(hex(k)+j[:-1],hashlib.sha256((chr(k)+j[:-1]).encode('utf-8')).hexdigest())
        print(j[:-1]+hex(k),hashlib.sha256((j[:-1]+chr(k)).lower().encode('utf-8')).hexdigest())
        print(hex(k)+j[:-1],hashlib.sha256((chr(k)+j[:-1]).lower().encode('utf-8')).hexdigest())
REIAHNEVIL-picotf@webshell:~$ python rainbow.py > table
REIAHNEVIL-picotf@webshell:~$
```

This redirected the output of the program into a file named **table**, which contained the precomputed hashes and their associated cheese names and salts.

The resulting table allowed me to quickly search for any hash returned by the challenge server.

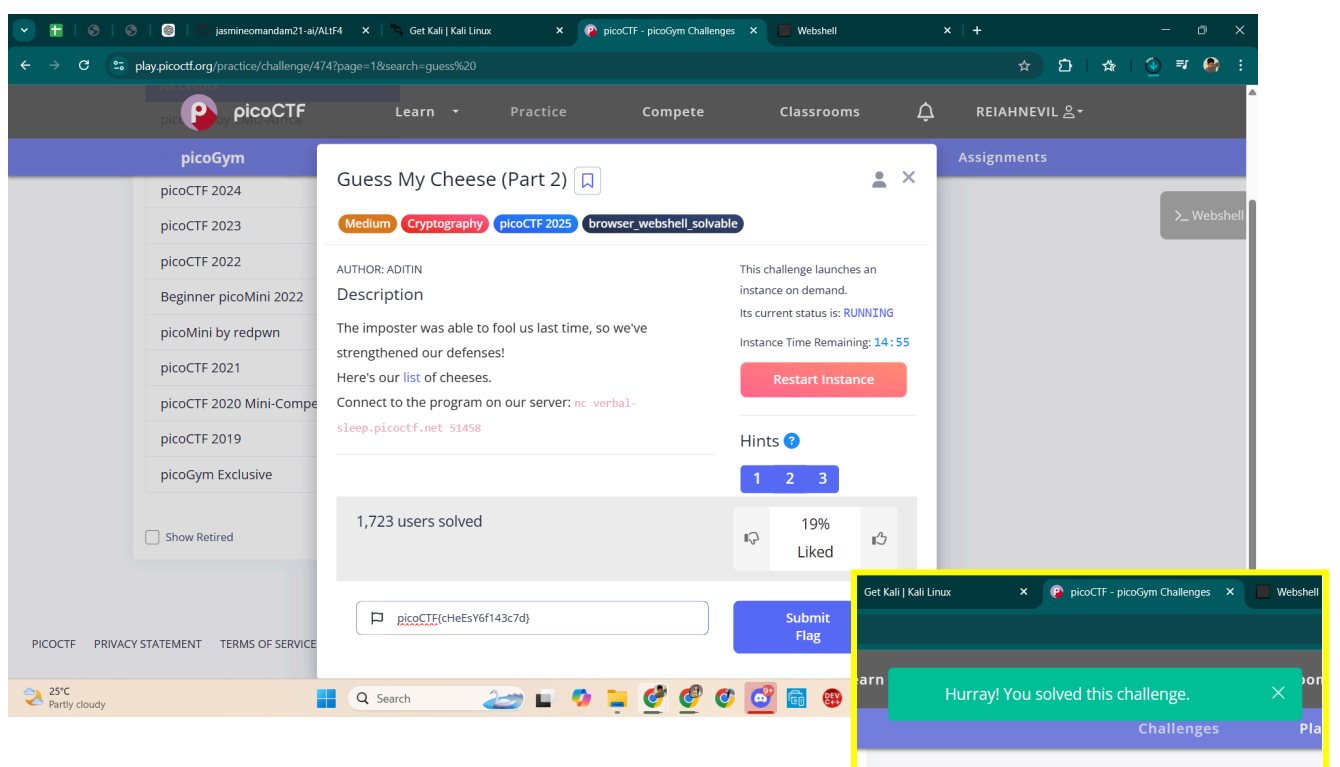
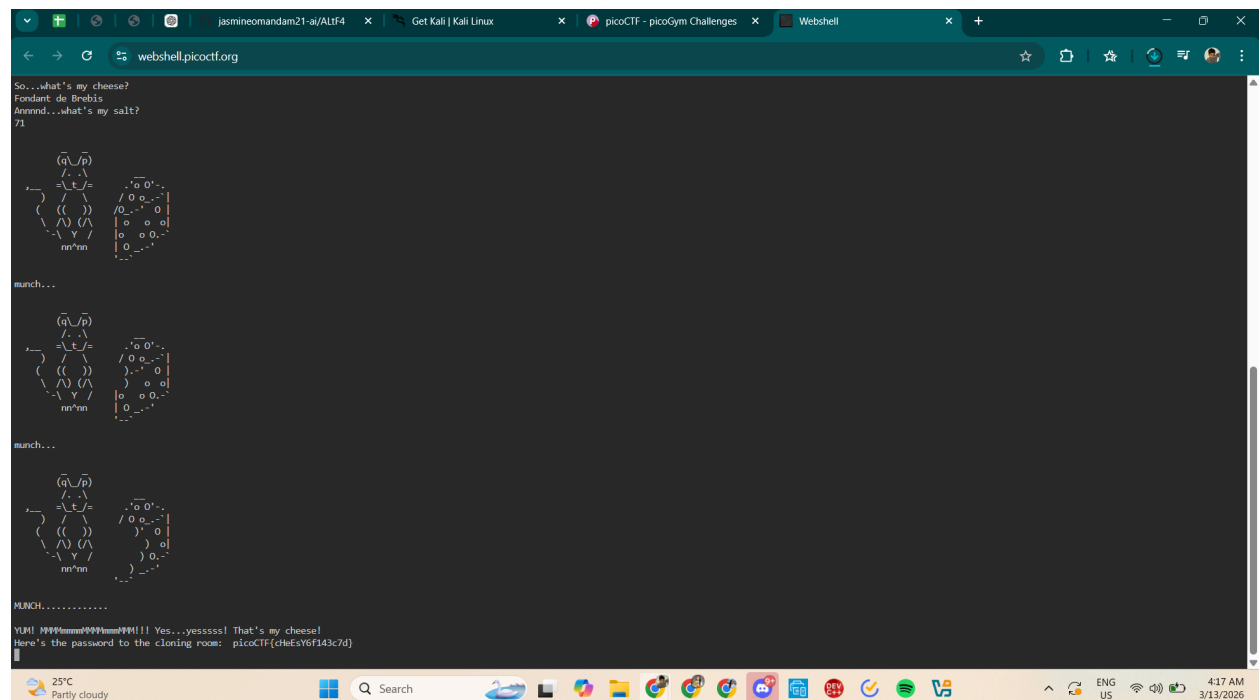
#### 4. Connecting to the Challenge Server

Next, I connected to the remote challenge server using netcat. The server generated a hash corresponding to a salted cheese name. I copied the hash and searched for it within the rainbow table using a search command. Once the matching hash was located, the corresponding entry revealed the correct cheese name and salt.

```
< -> +  
jasmineomandan21-at|LFid x Get Kali | Kali Linux x picoCTF - picoGym Challenges x Webshell  
  
webshell.picoctf.org ☆ ☆ ⌂ 🔍 👤 :  
REIAHNEVIL-picoctf@webshell:~$ nc verbal-sleep.picoctf.net 51458  
  
*****  
*** Part 2 ***  
*** The Mystery of the CLONED RAT ***  
*****  
  
[DRAT] The evil Dr. Lacktoes Inn Tolerant's clone was able to guess the cheese last time! I guess simple ciphers aren't good hashing methods. But now I've strengthened my encryption scheme so that now ONLY SQUEEYX can guess it...  
  
Here's my secret cheese -- if you're Squeeyx, you'll be able to guess it: 8e38ed31ae1c4ab47d0a33ba3d80bab7198b22928341bd9150339db594235  
  
Commands: (g)uess my cheese  
what would you like to do?  
?z  
[!] Stopped nc verbal-sleep.picoctf.net 51458  
REIAHNEVIL-picoctf@webshell:~$ cat table | grep 8e38ed31ae1c4ab47d0a33ba3d80bab7198b22928341bd9150339db594235  
hash: 8e38ed31ae1c4ab47d0a33ba3d80bab7198b22928341bd9150339db594235: command not found  
REIAHNEVIL-picoctf@webshell:~$ cat table | grep 8e38ed31ae1c4ab47d0a33ba3d80bab7198b22928341bd9150339db594235  
fondant de Brebisde51 8e38ed31ae1c4ab47d0a33ba3d80bab7198b22928341bd9150339db594235  
Fondant de Brebisde71 8e38ed31ae1c4ab47d0a33ba3d80bab7198b22928341bd9150339db594235  
REIAHNEVIL-picoctf@webshell:~$ fg  
nc verbal-sleep.picoctf.net 51458  
q  
  
((\sqrt{P}))  
(\sqrt{\frac{t}{n}})  
(((\sqrt{\frac{t}{nn}})))  
SQUEAK SQUEAK SQUEAK  
  
((\sqrt{P}))  
(\sqrt{\frac{t}{n}})  
(((\sqrt{\frac{t}{nn}})))  
REIAHNEVIL-picoctf@webshell:~$ fg  
nc verbal-sleep.picoctf.net 51458  
So...what's my cheese?
```

## 5. Identifying the Correct Answer

The rainbow table search revealed the correct combination. After submitting this combination to the challenge server, the system confirmed the correct answer and returned the flag.



### Reflection:

This challenge provided practical experience with hash-based cryptographic attacks, particularly the use of rainbow tables to reverse hashed values when the input space is limited.

Although SHA-256 is a secure cryptographic hash function, its security depends heavily on how it is used. In this challenge, the system applied only two hexadecimal characters of salt, resulting in just 256 possible salt values. Because the cheese names were also taken from a known list, the total number of possible inputs was small enough to precompute.

By generating a rainbow table containing all possible salted combinations, it became possible to quickly identify the original input corresponding to the given hash.

This exercise demonstrated that even strong hash functions like SHA-256 can be vulnerable if the input space is predictable or the salt size is too small. In real-world systems, developers mitigate this risk by using longer random salts and computationally expensive hashing algorithms, especially for sensitive data such as passwords.